

DCIT 308

Data Structures and Algorithms II

Lecture 6

Binary Search Trees and Searching Strategies

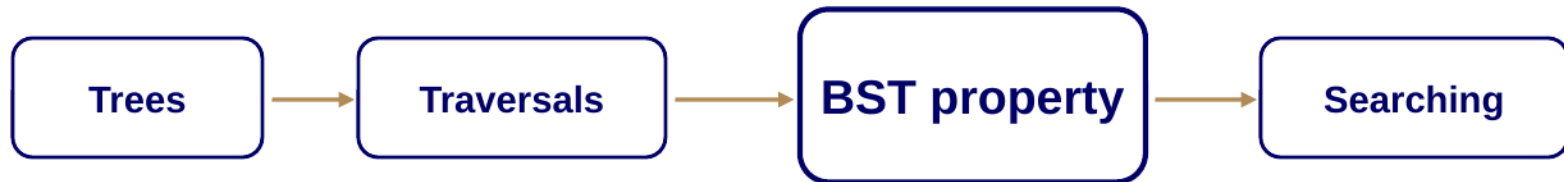


College of Basic and Applied Sciences
School of Physical and Mathematical Sciences
Department of Computer Science

Focus: ordered tree property, search path, insertion, deletion cases and Java implementation

Where this lecture fits

We have learned trees and traversal. Now we add ordering so the tree can support faster search.



A BST is a binary tree where values are arranged so search follows one branch at a time instead of visiting all nodes.

ordinary traversal: useful for visiting all nodes

BST search: useful for locating one key quickly

next topics: balancing trees to keep search efficient

Learning outcomes

By the end of this lecture, students should be able to:

- state and verify the Binary Search Tree ordering property
- trace successful and unsuccessful BST search paths accurately
- write recursive and iterative search algorithms for BSTs
- implement insertion and deletion cases while preserving the BST property
- compare linear search, binary search on arrays and BST search
- analyse best, average and worst-case time complexity using tree height

The goal is not only to draw a BST; it is to reason about why each move is left or right.

Motivation: why search structures matter

Real systems rarely search once. They search many times, so organisation matters.

Ghana-context examples

student index lookup in a class
list
national service PIN verification
patient folder number retrieval
library catalogue search
mobile-money transaction lookup

Search design question

**Can we organise records so
each comparison eliminates
many impossible answers?**

BSTs answer this by using order:
smaller keys go left, larger keys go
right.

Searching strategies at a glance

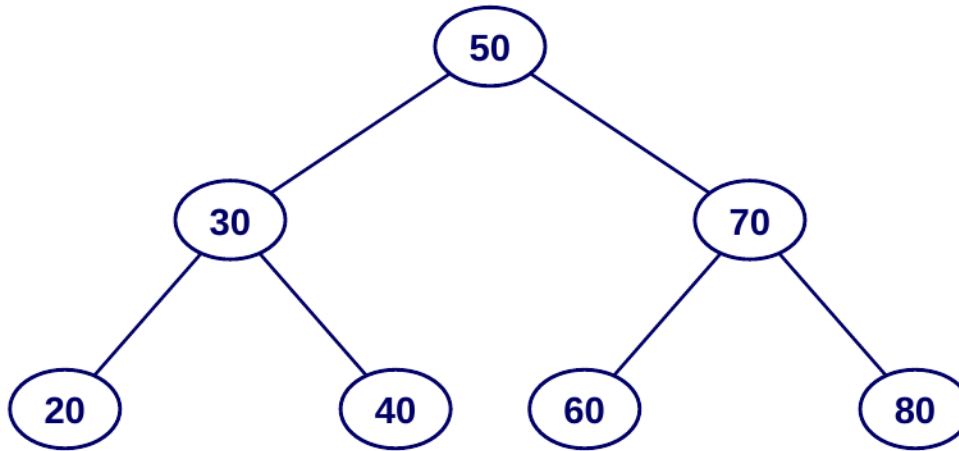
The same search problem can be solved in different ways depending on how data is stored.

Strategy	Data organisation	Typical cost	When it is useful
Linear search	unsorted list	$O(n)$	small data or one-off search
Binary search	sorted array	$O(\log n)$	static data, fast random access
BST search	ordered tree	$O(h)$	dynamic data: search + insert + delete

The BST cost depends on height h , so the shape of the tree is critical.

Binary Search Tree: formal property

For every node x in a BST:



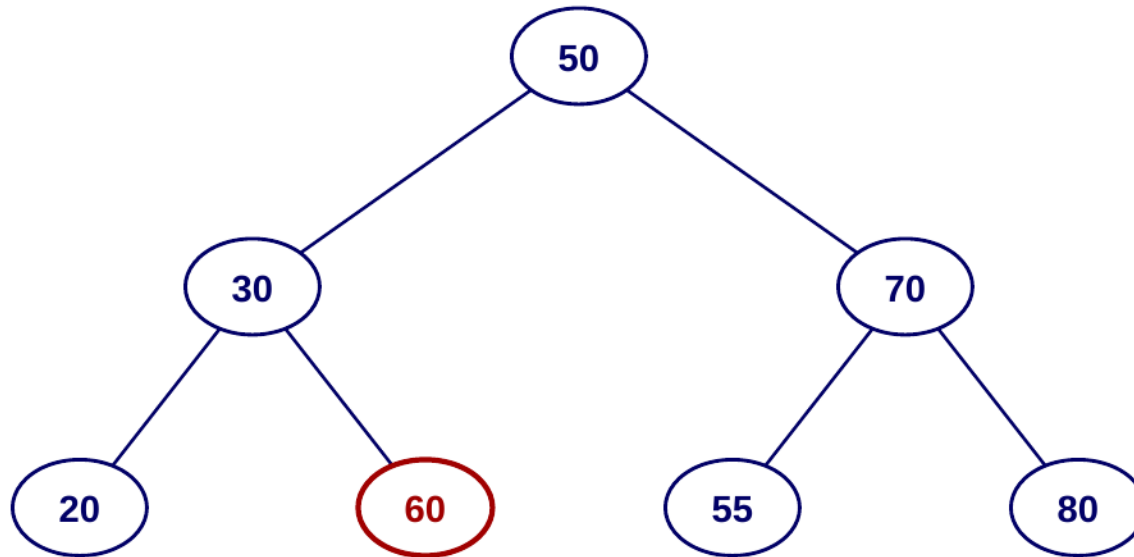
all keys in left subtree $< x.key$

all keys in right subtree $> x.key$

The property must hold at every node, not only at the root.

Common mistake: checking only the root is not enough

A tree can look reasonable but still violate the BST property in a lower subtree.

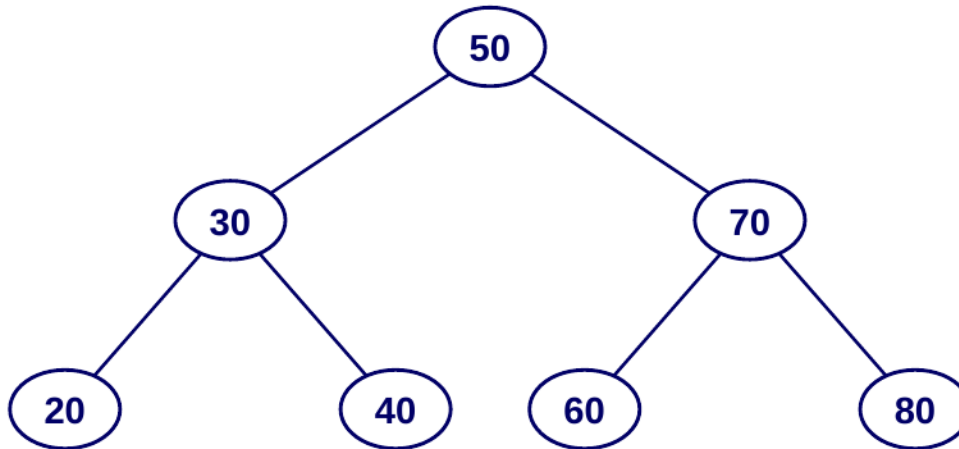


60 is in the left subtree of 50, so it must be < 50 . It is not.

BST verification needs bounds: each node must lie within its allowed min-max range.

BST property and inorder traversal

Inorder traversal of a valid BST visits keys in ascending order.



Inorder order:

20 30 40 50 60 70 80

If inorder is not sorted, the tree is not a valid BST.

BST search intuition: compare, then move

At each node, one comparison decides whether to stop, go left or go right.

$\text{target} < \text{node.key}$ → go left

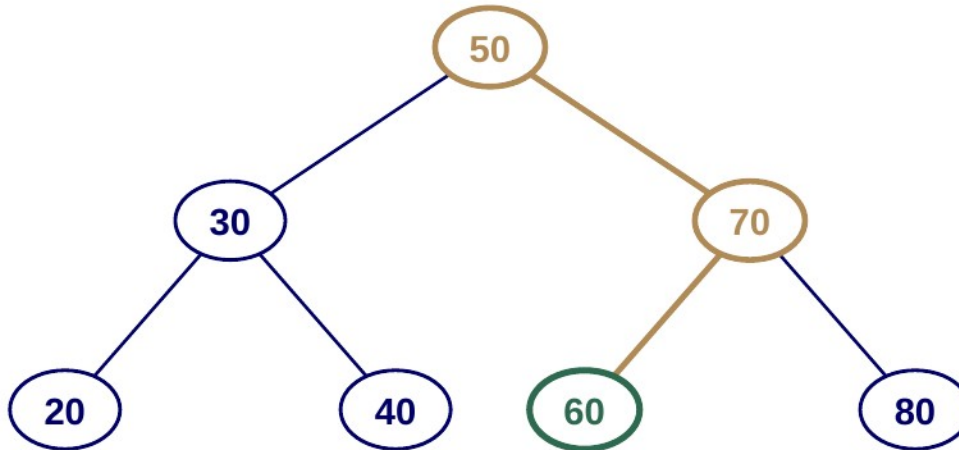
$\text{target} > \text{node.key}$ → go right

$\text{target} == \text{node.key}$ → found

**This strategy works
only because the tree
is ordered.**

Trace: successful search for 60

Search target = 60. Follow only one path.



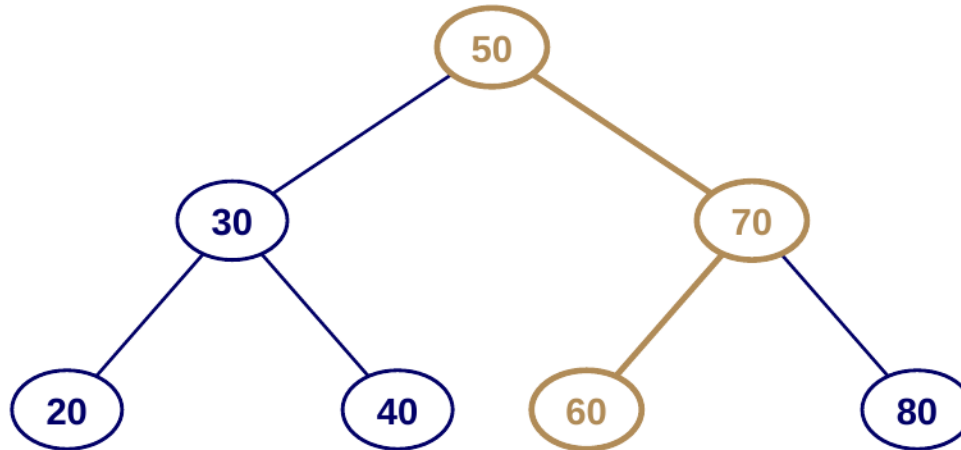
50 → 70 → 60

60 > 50, go right; 60 < 70, go left;
found 60.

Only 3 nodes are visited instead of all 7.

Trace: unsuccessful search for 65

Search target = 65. If the next required child is null, the key is absent.



$65 > 50 \rightarrow \text{right}$
 $65 < 70 \rightarrow \text{left}$
 $65 > 60 \rightarrow \text{right}$
right child is null

Result: NOT FOUND

Recursive BST search pseudocode

The recursive version searches a smaller subtree at every step.

```
BST_SEARCH(node, target)
  if node == null then
    return null

  if target == node.key then
    return node
  else if target < node.key then
    return BST_SEARCH(node.left,
target)
  else
    return BST_SEARCH(node.right,
target)
```

Base case

node is null
or key is found

Recursive case

search left subtree
or right subtree

At most one recursive call is made per level.

Iterative BST search pseudocode

The iterative version follows the same path using a loop instead of recursion.

```
BST_SEARCH_ITERATIVE(root, target)
    current = root

    while current != null do
        if target == current.key then
            return current
        else if target < current.key then
            current = current.left
        else
            current = current.right

    return null
```

Implementation note

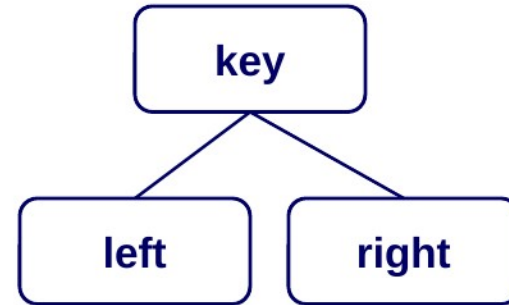
Iterative search avoids recursion-stack overhead but has the same comparison path.

Both versions are $O(h)$, where h is tree height.

Java implementation: BST node class

A BST node stores a key and references to left and right children.

```
class TreeNode {  
    int key;  
    TreeNode left;  
    TreeNode right;  
  
    TreeNode(int key) {  
        this.key = key;  
        this.left = null;  
        this.right = null;  
    }  
}
```



References connect nodes; there is no automatic next pointer as in a linked list.

Java implementation: search method

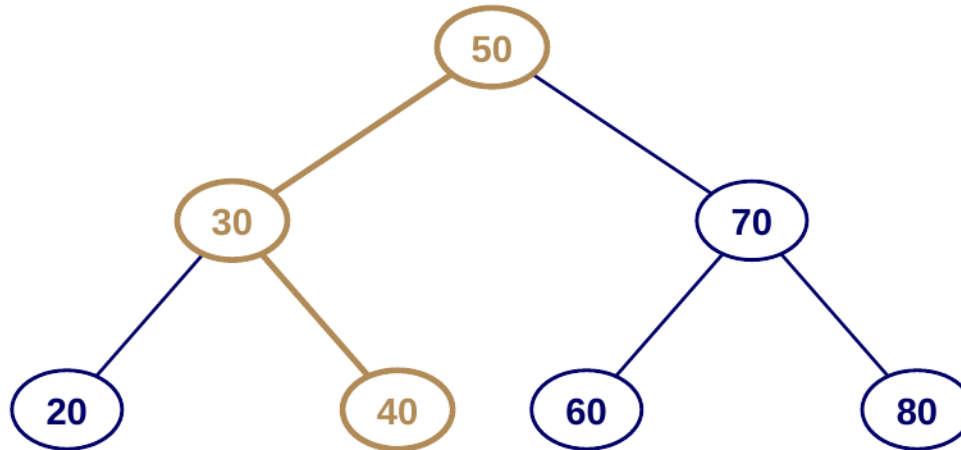
Use the BST property to choose one direction at each comparison.

```
public TreeNode search(TreeNode node, int target) {  
    if (node == null) {  
        return null;  
    }  
    if (target == node.key) {  
        return node;  
    }  
    if (target < node.key) {  
        return search(node.left, target);  
    }  
    return search(node.right, target);  
}
```

Trace the code with target 60 before running it. The method should call search on 50, then 70, then 60.

Insertion: search for the correct null position

Insertion follows the same comparison path as search, then attaches the new key as a leaf.



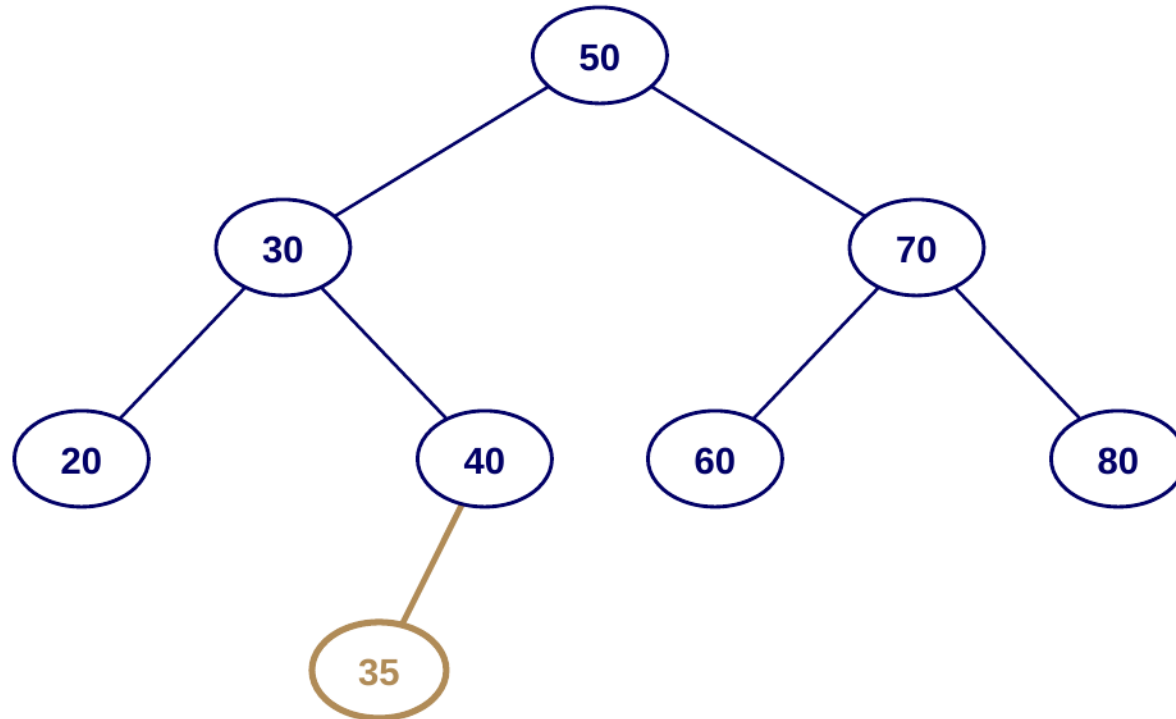
Insert 35

$35 < 50 \rightarrow$ left
 $35 > 30 \rightarrow$ right
 $35 < 40 \rightarrow$ left
attach at null

New nodes are inserted as leaves in a basic BST.

Insertion result: 35 added correctly

After insertion, the BST property must still hold at every node.



Check: 35 is greater than 30 but smaller than 40, so it belongs as the left child of 40.

Insertion pseudocode

A recursive insert returns the new root of the subtree after insertion.

```
BST_INSERT(node, key)
    if node == null then
        return new Node(key)

    if key < node.key then
        node.left = BST_INSERT(node.left,
key)
    else if key > node.key then
        node.right = BST_INSERT(node.right,
key)
    else
        ignore duplicate key

    return node
```

Why return node?

The parent must
reconnect to the possibly
changed subtree root.

Duplicate policy must be stated clearly: ignore, count, or store duplicates consistently.

Java implementation: insert method

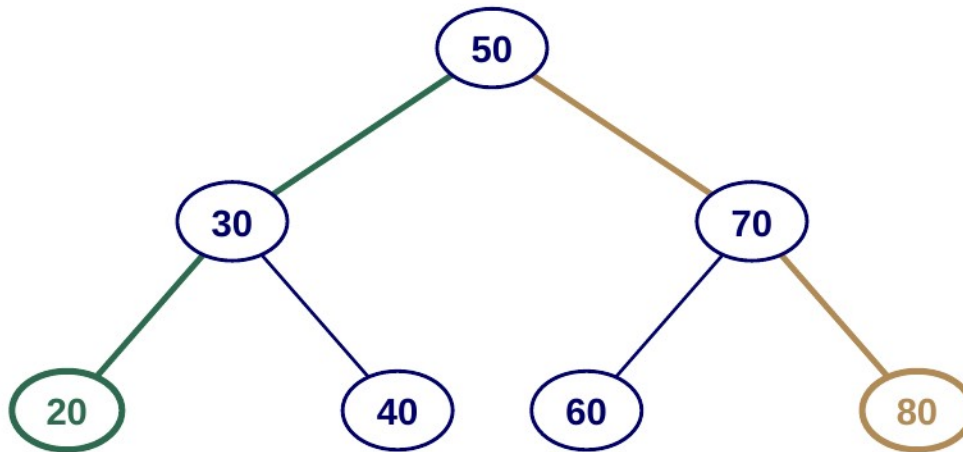
This version ignores duplicate keys and returns the subtree root.

```
public TreeNode insert(TreeNode node, int key) {
    if (node == null) {
        return new TreeNode(key);
    }
    if (key < node.key) {
        node.left = insert(node.left, key);
    } else if (key > node.key) {
        node.right = insert(node.right, key);
    }
    return node;
}
```

Use this method by assigning `root = insert(root, key)`.

Minimum and maximum search

The smallest key is found by following left pointers. The largest key is found by following right pointers.



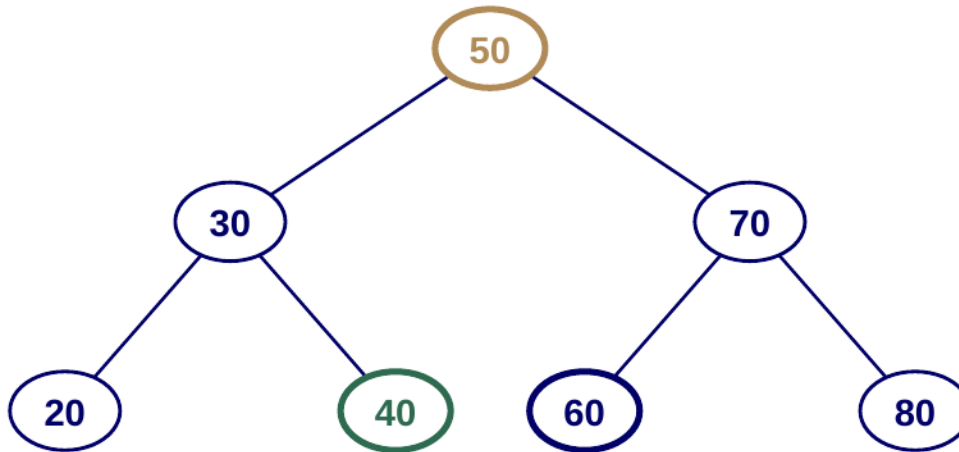
min = 20

max = 80

Both operations cost $O(h)$.

Predecessor and successor

Predecessor is the next smaller key; successor is the next larger key.



For key 50:

predecessor = max(left subtree) = 40

successor = min(right subtree) = 60

These ideas are important for deletion when a node has two children.

Deletion: three cases

Deletion is harder than search or insertion because the tree must remain a valid BST.

Case 1

delete a leaf
remove it directly

Case 2

delete node with one
child
replace by child

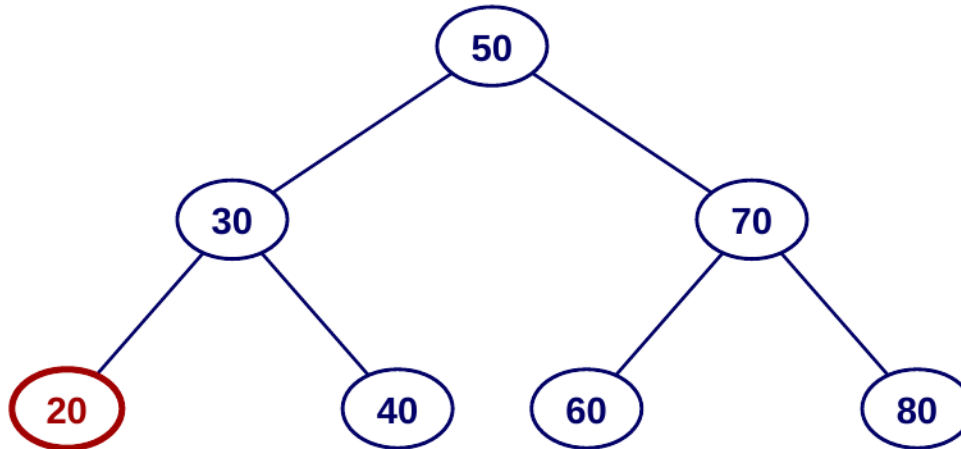
Case 3

delete node with two
children
replace by successor

We consider the cases separately so implementation becomes controlled and predictable.

Deletion case 1: remove a leaf

Deleting a leaf simply removes the parent pointer to that node.

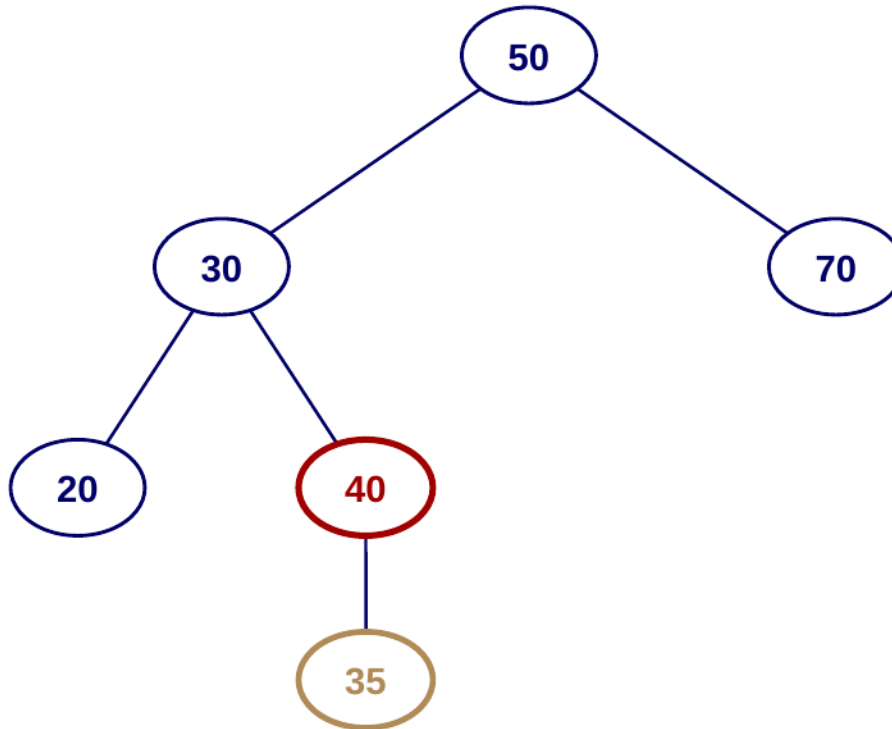


Delete 20

20 has no children, so parent 30.left becomes null.

Deletion case 2: node with one child

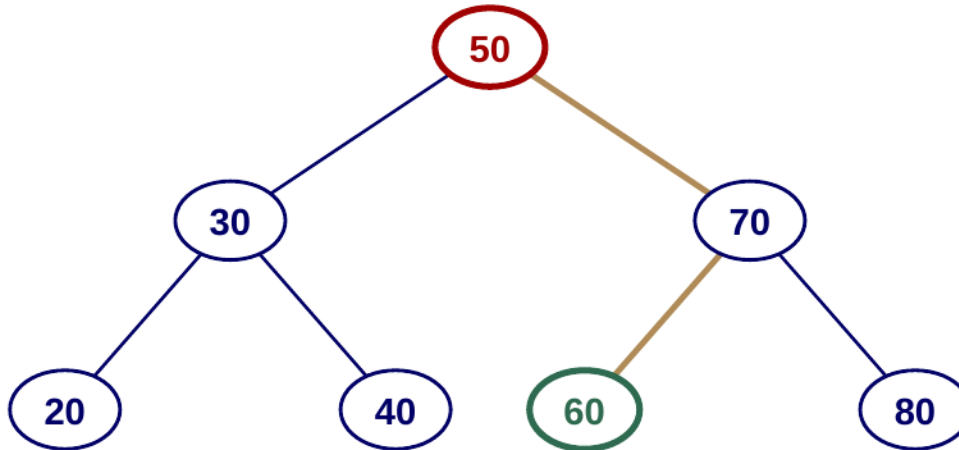
If the node has one child, connect its parent directly to that child.



Delete 40: replace 40 by its only child 35.

Deletion case 3: node with two children

Replace the key with its inorder successor, then delete the successor from the right subtree.



Delete 50

Successor = 60
copy 60 into root
remove original 60

Choosing the smallest key in the right subtree preserves BST order.

Deletion pseudocode

The general delete method handles all three cases.

```
BST_DELETE(node, key)
  if node == null then return null

  if key < node.key then
    node.left = BST_DELETE(node.left, key)
  else if key > node.key then
    node.right = BST_DELETE(node.right, key)
  else
    if node.left == null then return node.right
    if node.right == null then return node.left
    succ = MIN(node.right)
    node.key = succ.key
    node.right = BST_DELETE(node.right, succ.key)

  return node
```

The recursive return reconnects each subtree correctly.

Complexity: everything depends on height

BST operations search down one root-to-leaf path.

Search

$O(h)$

Insert

$O(h)$

Delete

$O(h)$

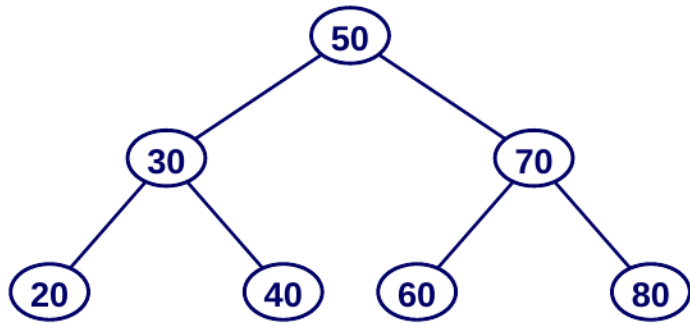
Best / balanced: $h \approx \log_2 n$

Worst / skewed: $h \approx n$

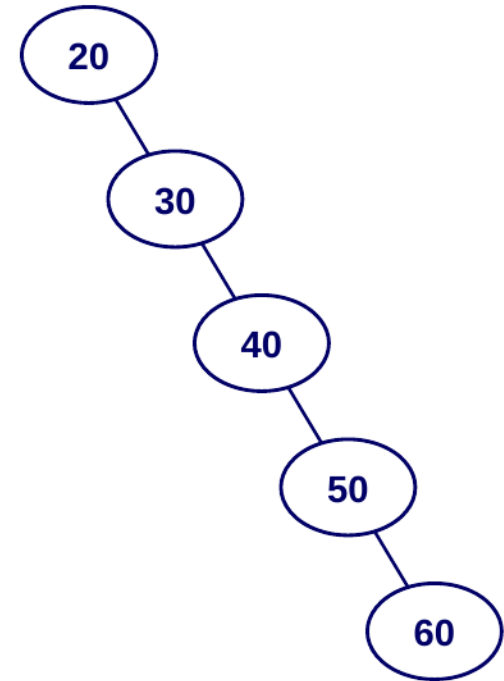
A BST is only efficient when its height remains small.

Tree shape matters: same keys, different cost

Insertion order can create a balanced-looking tree or a chain.



Good shape: height 2



Bad shape: chain

Sorted insertion order often produces the bad case in a basic BST.

Choosing a searching strategy

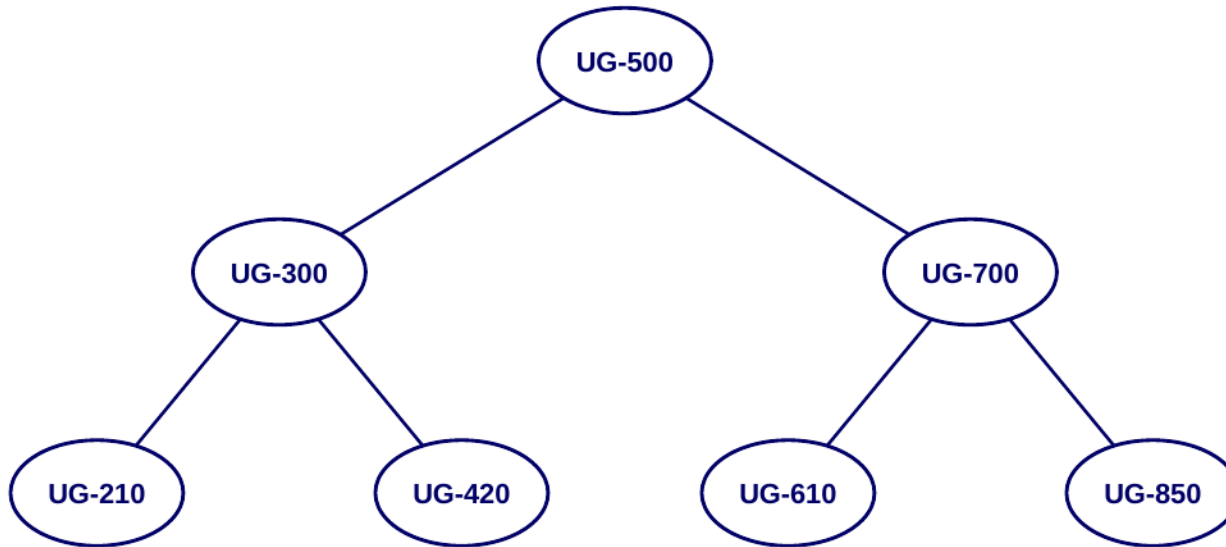
The data structure should match the type of search and update workload.

Scenario	Suggested strategy	Reason
One-time search in small attendance list	linear search	simplest approach is enough
Many searches in a static sorted list	binary search	array order and random access help
Frequent search plus insertion/deletion	BST or balanced tree	dynamic updates are easier
Database range query over ordered keys	tree-based index	supports ordered navigation

Decision rule: consider data size, update frequency, ordering needs and worst-case guarantees.

Real-world analogy: index lookup

A BST behaves like an organised index: each comparison narrows the possible section.



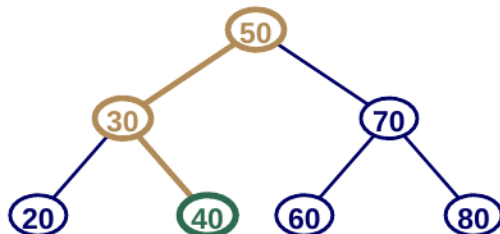
Example: student folder numbers or library catalogue IDs can be searched by comparing keys.

In real production systems, balanced tree variants are often preferred to avoid bad shapes.

Trace table: search for 40

Always trace comparison decisions to avoid ambiguous explanations.

Step	Current key	Compare with 40	Move
1	50	$40 < 50$	left
2	30	$40 > 30$	right
3	40	$40 == 40$	found



Trace tables are useful for debugging and for explaining your algorithm in assessments.

In-class activity: build and search a BST

Work in pairs. Insert the keys in order, then trace the search paths.

Keys to insert:

45, 25, 65, 15, 35, 55, 75, 30

Tasks

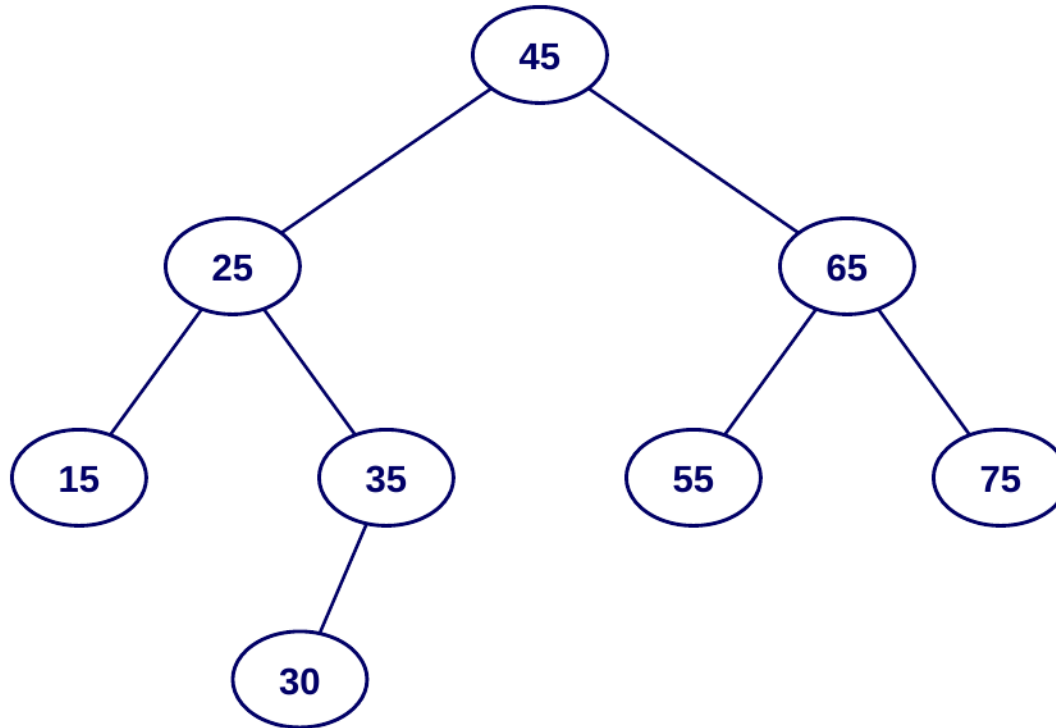
- draw the final BST to scale
- trace search for 55
- trace unsuccessful search for 50
- state the height of the final tree
- give the inorder traversal result

Challenge

Which insertion order would make this set more skewed? Explain why.

Activity solution: one correct final tree

Final tree after inserting 45, 25, 65, 15, 35, 55, 75, 30.



Search 55: 45 → 65 → 55

Search 50: 45 → 65 → 55 → null

Inorder: 15, 25, 30, 35, 45, 55, 65, 75

Mini-lab: implement BST search and insertion

Goal: implement a BST and prove the output using trace evidence.

Part A: Build

- create TreeNode class
- write insert method
- build from int array

Part B: Search

- write recursive search
- write iterative search
- print search path

Part C: Analyse

- state height
- state Big-O
- explain shape effect

Required submission: Java file, screenshot of outputs, hand-drawn tree, and one-page explanation.

Common mistakes to avoid

Most BST errors come from breaking the ordering property or mishandling null references.

checking only the root property and ignoring subtree bounds
forgetting to return and reconnect subtree roots during
insertion/deletion
treating a BST as automatically balanced
using inorder sorted output as proof without checking duplicate policy
forgetting the null base case in recursive search
deleting a two-child node without using predecessor or successor

Debugging tip: after every insert or delete, print inorder traversal and check that it is sorted.

What students should now know

a BST is a binary tree with ordered left and right subtrees
search follows one root-to-leaf path using comparisons
search, insert, min, max and delete are $O(h)$
balanced trees give logarithmic height; skewed trees can degrade to linear height
insertion order strongly affects basic BST shape
deletion has three cases: leaf, one child and two children

Next lecture: balanced trees and why guaranteed logarithmic performance matters.

References and further reading

Core references used to prepare this lecture:

MIT OpenCourseWare 6.006. Binary Search Trees and BST Sort; Binary Trees lectures.

Sedgewick, R., & Wayne, K. Algorithms, 4th Edition. Princeton Algorithms: 3.2 Binary Search Trees.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms. BST chapter.

OpenDSA. Binary Search Trees: find, insert, delete, height and tree shape.

Stanford CS166. Advanced data structures and balanced search tree perspectives.

Recommended practice: draw each tree and trace the exact comparison path before coding.